ISLAMIC UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering

Software Maintenance Lab

Project: Math Problem Solver

Student Name:	Muhammad Yasir Raiyan
Student ID:	210042152
Course Title:	Software Maintenance Lab
Course Code:	SWE-4802
Department:	Computer Science and Engineering
Prog:	B.S.C IN S.W.E
Semester:	8
Academic Year:	2024-25

Academic Year: 2026

Contents

Abstract

Software maintenance is an essential phase of the software development life cycle because software systems must continuously evolve after their initial development. Defects must be corrected, software may need to operate in a new environment, internal quality may need improvement, and users may require additional functionality.

This report presents the maintenance and analysis of a Python-based application called *Math Problem Solver*. The project was developed as a Software Maintenance project and demonstrates four major categories of software maintenance: corrective, adaptive, preventive, and perfective maintenance.

Corrective maintenance was performed to address a division-by-zero problem. Adaptive maintenance introduced a Flask-based web interface to extend the command-line application to a web environment. Preventive maintenance improved input validation, type information, error handling, code organization, and maintainability. Perfective maintenance extended the mathematical functionality of the system by supporting operations such as factorial and percentage.

In addition, four transformation and software-analysis tools were applied to the project: **Loguru**, **PySnooper**, **VizTracer**, and **SnakeViz**. These tools provide different forms of information about the software. Loguru provides application logging, PySnooper provides detailed execution tracing, VizTracer provides runtime execution visualization, and SnakeViz visualizes Python profiling information.

The final project was verified using Python's `unittest` framework. The final test execution successfully passed 16 automated tests. The project therefore demonstrates how maintenance activities, testing, debugging, tracing, logging, and performance analysis can be combined in a practical software maintenance workflow.

1. Introduction

1.1 Background

Software maintenance refers to the activities performed after the initial development of a software system to keep the system functional, reliable, maintainable, and useful.

A software system may require maintenance for several reasons. Existing defects may need to be corrected, the execution environment may change, the internal implementation may need improvement, or users may request new functionality.

The project used in this laboratory is the **Math Problem Solver**, a Python-based mathematical calculation application. The project was progressively maintained and extended through different maintenance activities.

The project also applies software analysis and transformation tools to understand the behavior and performance of the maintained software.

1.2 Project Objectives

The main objectives of the project are:

- To develop and maintain a mathematical calculation application.
- To demonstrate different categories of software maintenance.
- To identify and correct software defects.
- To adapt the application to a web-based environment.
- To improve code quality and maintainability.
- To add new mathematical features.
- To apply software analysis and transformation tools.
- To understand program execution and performance.
- To perform automated testing after maintenance activities.

1.3 Assignment Objectives

The laboratory assignment focuses on the following objectives:

- Introduction to transformation and software analysis tools.
- Application of the tools to a working software project.
- Explanation of the insights obtained from tool outputs.
- Identification of maintenance scenarios where the tools can be used.
- Identification of alternative tools.
- Selection of preferred tools for future software projects.

2. Project Overview

2.1 Math Problem Solver

The Math Problem Solver is a Python-based mathematical calculation application developed as a Software Maintenance project.

The original system was primarily command-line based. Through successive maintenance activities, the project was improved and extended.

The current project demonstrates:

- Corrective Maintenance
- Adaptive Maintenance
- Preventive Maintenance
- Perfective Maintenance
- Program Comprehension
- Change Management
- Impact Analysis
- Reverse Engineering
- Refactoring
- Automated Testing
- Logging
- Execution Tracing
- Performance Profiling

2.2 Technologies Used

The project uses the following technologies and tools:

Table 2.1: Technologies and Tools Used in the Project

Technology / Tool	**Purpose**
Python	Main programming language
Flask	Web-based adaptation
unittest	Automated testing
Loguru	Application logging
PySnooper	Debugging and execution tracing
VizTracer	Program execution tracing
cProfile	Python performance profiling
SnakeViz	Visualization of profiling results
Git	Version control
GitHub	Source-code repository
VS Code	Development environment

These technologies are documented in the current project repository. :contentReference[oaicite:2]index=2

2.3 Project Structure

The current repository contains separate components for calculation, testing, analysis, web presentation, utilities, and documentation.

The main project structure is:

```
Math-Problem-Solver/
|
+-- Calculator/
|   +-- basic.py
|   +-- advanced.py
|   +-- validator.py
|
+-- Commands/
|
+-- Report/
|
+-- analysis/
|   +-- logging_demo.py
```

```
|   +-- pysnooper_demo.py
|   +-- viztrace_demo.py
|   +-- profile_demo.py
|
+-- templates/
|   +-- index.html
|
+-- tests/
|   +-- test_basic.py
|   +-- test_advaned.py
|   +-- test_integration.py
|   +-- test_validator.py
|
+-- utils/
|   +-- formatter.py
|   +-- logger.py
|
+-- app.py
+-- main.py
+-- Requirements.txt
+-- README.md
+-- LICENSE
```

The repository's current main branch lists the `Calculator`, `Commands`, `Report`, `analysis`, `templates`, `tests`, and `utils` directories, along with the application and configuration files. :contentReference[oaicite:3]index=3

2.4 Mathematical Functionality

The final system supports the following mathematical operations:

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Power
6. Square Root
7. Cube Root

8. Factorial

9. Percentage

The repository documents these nine final mathematical operations. :contentReference[oaicite:4]index=4

3. Software Maintenance Lifecycle

3.1 Maintenance Evolution

The project follows the following maintenance sequence:

$$V1 \rightarrow V1.1 \rightarrow V1.2 \rightarrow V1.3 \rightarrow V2.0$$

The individual stages are:

1. **V1**: Initial baseline system.
2. **V1.1**: Corrective maintenance.
3. **V1.2**: Adaptive maintenance.
4. **V1.3**: Preventive maintenance.
5. **V2.0**: Perfective maintenance and final system.

3.2 Maintenance Lifecycle Diagram

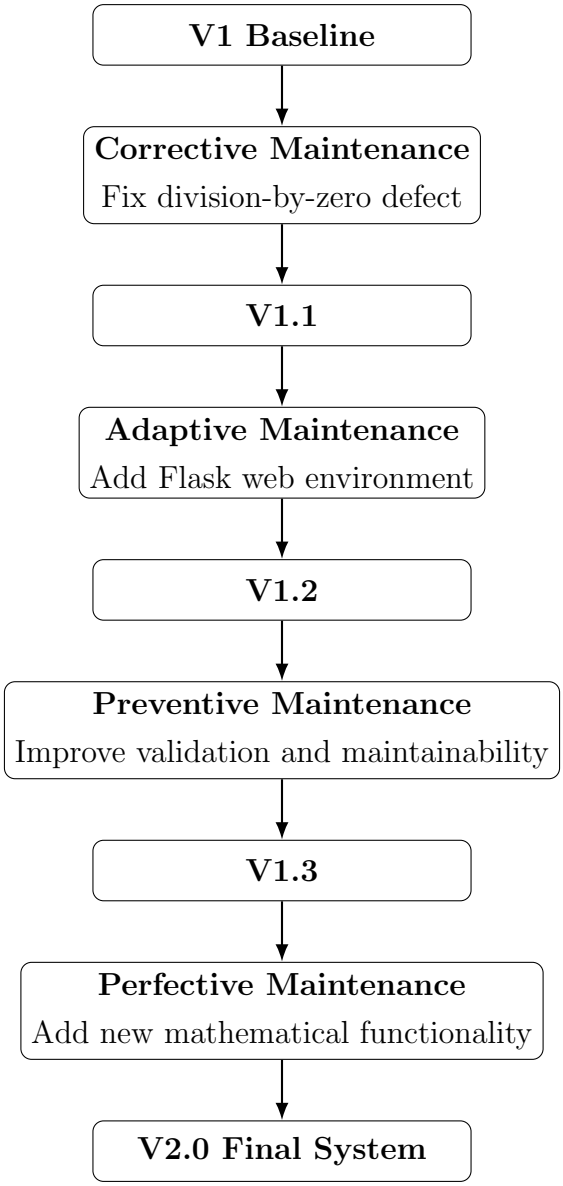

Figure 3.1: Software Maintenance Evolution

4. Corrective Maintenance

4.1 Definition

Corrective maintenance is performed to identify and fix defects or errors found in existing software.

4.2 Problem Identified

A defect was identified in the division functionality.

When the denominator was zero, the application could produce a `ZeroDivisionError`. This could terminate the operation unexpectedly.

4.3 Program Comprehension

The original calculation flow was analyzed as follows:

```
User
 |
 v
main.py
 |
 v
Calculator
 |
 v
divide(a,b)
 |
 v
a / b
```

The `divide()` function was identified as the location that required correction.

4.4 Change Management

The requested change was:

> Prevent division by zero and provide a meaningful error message.

The function was modified to validate the denominator before performing the division.

```
def divide(a: float, b: float) -> float:
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

4.5 Impact Analysis

The main affected components were:

- Calculator division functionality – direct impact.
- Main application – indirect impact.
- Basic calculator test module – regression testing.

4.6 Reverse Engineering

The original execution flow was reconstructed as:

```
User
 |
 v
Select Division
 |
 v
main.py
 |
 v
divide(a,b)
 |
 v
Denominator = 0
 |
 v
ZeroDivisionError
```

After maintenance, the flow became:

```
User
 |
 v
Select Division
 |
 v
divide(a,b)
 |
 v
Check denominator
 |
 +---- b == 0 ----> ValueError
 |
 +---- b != 0 ---> a / b
```

4.7 Regression Test

A regression test was added:

```
def test_divide_by_zero(self):
    with self.assertRaises(ValueError):
        divide(10, 0)
```

4.8 Testing Result

Before corrective maintenance:

10 tests passed

After corrective maintenance:

11 tests passed

Therefore, the corrective maintenance was verified through automated testing. The repository documents this transition from 10 to 11 successful tests. :contentReference[oaicite:5]index=5

4.9 Version Evolution

$V1 \rightarrow$ Corrective Maintenance $\rightarrow V1.1$

5. Adaptive Maintenance

5.1 Definition

Adaptive maintenance modifies software so that it can work in a changed environment or support new environmental requirements.

5.2 Problem

The original application was primarily command-line based.

The system was required to support access through a web browser.

5.3 Maintenance Action

A Flask-based web interface was introduced while reusing the existing calculator logic.

The adapted architecture became:

```
Browser
 |
 v
index.html
 |
 v
app.py
 |
 v
Validation
 |
 v
Calculator Functions
 |
 v
Result
 |
```

```
 v
Browser
```

5.4 Technology Added

The adaptive maintenance introduced:

- Flask
- HTML web interface
- Web application routing

5.5 Impact Analysis

The main affected components were:

- `app.py` – web application.
- `Requirements.txt` – dependency configuration.
- `templates/index.html` – web interface.

The existing calculator modules were reused rather than rewritten.

5.6 Benefits

The adaptive maintenance provides:

- Browser-based access.
- Improved user interaction.
- Web-based execution.
- Better accessibility.
- Adaptation to a new execution environment.

5.7 Version Evolution

$$V1.1 \rightarrow \text{Adaptive Maintenance} \rightarrow V1.2$$

The repository explicitly documents Flask as the web adaptation technology and describes reuse of the existing calculator modules. :contentReference[oaicite:6]index=6

6. Preventive Maintenance

6.1 Definition

Preventive maintenance improves the internal quality of software to reduce the possibility of future failures.

6.2 Maintenance Activities

The project was improved through:

- Better input validation.
- Type hints.
- Improved error handling.
- Cleaner code organization.
- Improved maintainability.

For example, type hints make the expected input and output clearer:

```
1 def add(a: float, b: float) -> float:
2     return a + b
```

6.3 Benefits

The preventive maintenance activities help to:

- Reduce future errors.
- Improve code readability.
- Improve maintainability.
- Make debugging easier.
- Improve software reliability.

6.4 Version Evolution

$$V1.2 \rightarrow \text{Preventive Maintenance} \rightarrow V1.3$$

The current repository identifies input validation, type information, error handling, and maintainability as the main preventive-maintenance areas. :contentReference[oaicite:7]index=7

7. Perfective Maintenance

7.1 Definition

Perfective maintenance improves existing software by adding useful features or improving functionality based on user requirements.

7.2 Maintenance Action

Additional mathematical functionality was introduced to make the application more useful.

The final system supports:

- Addition
- Subtraction
- Multiplication
- Division
- Power
- Square Root
- Cube Root
- Factorial
- Percentage

7.3 Factorial

Factorial can be represented mathematically as:

$$n! = n(n-1)(n-2)\cdots 2(1)$$

For example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

The factorial feature requires appropriate validation because factorial is not defined for negative integers.

7.4 Percentage

The percentage operation can be represented as:

$$\text{Percentage} = \frac{\text{Value} \times \text{Percent}}{100}$$

For example:

$$20\% \text{ of } 500 = \frac{500 \times 20}{100} = 100$$

7.5 Benefits

The additional functionality provides:

- More mathematical operations.
- Greater usefulness to users.
- Improved application capability.
- A more complete mathematical calculator.

7.6 Testing

After adding the new functionality, the complete test suite was executed.

The final result was:

```
1 ................
2 ----------------------------------------------------------------------
3 Ran 16 tests in 0.002s
4
5 OK
```

7.7 Version Evolution

$$V1.3 \rightarrow \text{Perfective Maintenance} \rightarrow V2.0$$

The current repository documents the final nine mathematical operations and the 16-test result. :contentReference[oaicite:8]index=8

8. Transformation and Analysis Tools

8.1 Overview

The laboratory required the practical application of four tools:

1. Loguru
2. PySnooper
3. VizTracer
4. SnakeViz

The tools provide different types of information.

Table 8.1: Transformation and Analysis Tools

Tool	**Primary Purpose**	**Main Output**
Loguru	Logging and monitoring	Application events and errors
PySnooper	Debugging and tracing	Variables and execution flow
VizTracer	Runtime execution tracing	Visual execution timeline
SnakeViz	Profiling visualization	Performance information

The four tools are explicitly documented in the current repository. :contentReference[oaicite:9]index=9

9. Loguru

9.1 Introduction

Loguru is a Python logging library that provides a simple way to generate readable application logs.

In the Math Problem Solver project, Loguru is used to monitor mathematical operations and application events.

9.2 Usage in the Project

A typical logging implementation can record the start and result of a calculation:

```
from loguru import logger

logger.add("logs/math_solver.log")

logger.info("Starting mathematical calculation")

result = add(4, 3)

logger.info("Addition result: {}", result)
```

The project analysis records an example similar to:

```
[LOG] Addition operation performed
Addition result: 7.0
```

9.3 Insight from Output

The Loguru output provides a chronological record of application activities.

From the example output, we can determine:

1. The addition operation was executed.

2. The operation completed successfully.
3. The resulting value was 7.0.

Therefore, Loguru answers the question:

What happened during application execution?

9.4 Maintenance Application

Loguru is particularly useful during corrective maintenance.

Suppose a user reports that an operation is producing an unexpected result. Logs can help the developer determine:

- Which operation was executed.
- When the operation occurred.
- What result was produced.
- Whether an error occurred.

This reduces the need to reproduce every reported problem manually.

9.5 Project Execution

From the project root:

```
python analysis\logging_demo.py
```

The repository documents this command for the Loguru demonstration. :contentReference[oaicite:10]index=10

10. PySnooper

10.1 Introduction

PySnooper is a Python debugging and execution-tracing tool.

It automatically records information about the execution of a function, including variable values and return values.

10.2 Usage in the Project

A mathematical function can be traced using:

```
import pysnooper

@pysnooper.snoop()
def calculate(a, b):
    result = a + b
    return result
```

A representative trace can contain:

```
Starting var: a = 4
Starting var: b = 3
New var: result = 7
Return value: 7
```

10.3 Insight from Output

The output allows the developer to understand:

- Which variables were created.
- The values stored in variables.
- How execution moved through the function.

- What value was returned.

Therefore, PySnooper answers:

How did the function execute internally?

10.4 Maintenance Application

PySnooper is useful during corrective maintenance.

For example, if a percentage calculation returns an unexpected value, the developer can trace the function and inspect its intermediate variables.

This is especially helpful for understanding unfamiliar or legacy code.

10.5 Project Execution

From the project root:

```
python analysis\pysnooper_demo.py
```

The repository specifies this command for the PySnooper analysis. :contentReference[oaicite:11]index=11

11. VizTracer

11.1 Introduction

VizTracer is a Python execution-tracing tool that records program execution and allows developers to visualize the execution flow.

It is useful when the developer needs more than textual logging.

11.2 Usage in the Project

The project contains a VizTracer demonstration script.

The command used is:

```
viztracer analysis\viztrace_demo.py
```

VizTracer generates trace information that can be inspected using its visualization interface.

11.3 Information Provided

The visualization can show:

- Function calls.
- Execution sequence.
- Execution duration.
- Repeated operations.
- Program execution flow.

A simplified execution relationship can be represented as:

```
main()
 |
 +---- add()
```

```
|
+---- logger
|
+---- result
```

11.4 Insight from Output

VizTracer provides a dynamic view of program behavior.

It helps answer:

When and in what order did different parts of the program execute?

This is useful for understanding interactions between application components.

11.5 Maintenance Application

VizTracer can support corrective and preventive maintenance.

For example, suppose a future version of the web application becomes slower. The execution trace can help identify:

- Repeated function calls.
- Long-running functions.
- Unexpected execution paths.
- Functions requiring optimization.

11.6 Project Execution

If necessary, VizTracer can be installed using:

```
pip install viztracer
```

Then the demonstration can be executed using:

```
viztracer analysis\viztrace_demo.py
```

These commands are documented in the current repository. :contentReference[oaicite:12]index=12

12. SnakeViz

12.1 Introduction

SnakeViz is a visualization tool for Python profiling information generated by `cProfile`.

Unlike ordinary logging, profiling focuses on execution statistics and performance.

12.2 Profiling Process

The project first generates profiling data:

```
python -m cProfile -o profile.prof analysis\profile_demo.py
```

The resulting profile can then be visualized with:

```
snakeviz profile.prof
```

12.3 Insight from Output

SnakeViz provides visual information about:

- Number of function calls.
- Total execution time.
- Cumulative execution time.
- Functions consuming significant execution time.
- Potential performance bottlenecks.

Therefore, SnakeViz answers:

Where is execution time being spent?

12.4 Maintenance Application

SnakeViz is particularly useful for preventive and perfective maintenance.

For example, after adding new mathematical features, profiling can be used to determine whether any function has introduced unnecessary execution overhead.

A developer can then focus optimization efforts on the functions that require improvement.

12.5 Project Execution

If necessary, SnakeViz can be installed using:

```
pip install snakeviz
```

Then:

```
python -m cProfile -o profile.prof analysis\profile_demo.py
snakeviz profile.prof
```

The current repository documents this profiling workflow. :contentReference[oaicite:13]index=13

13. Analysis of Tool Outputs

13.1 Different Types of Insights

The four tools provide complementary information.

Table 13.1: Output Insights

Tool	**Insight**
Loguru	What application event happened and what result/error was recorded.
PySnooper	How a function executed and how its variables changed.
VizTracer	When functions executed, in what order, and for how long.
SnakeViz	Which functions consumed execution time and where potential bottlenecks exist.

13.2 Conceptual Relationship

The tools can be summarized as:

Loguru → What happened?

PySnooper → How did it happen?

VizTracer → When and in what order?

SnakeViz → Where is time spent?

Together, these tools provide a broader understanding of the existing system.

13.3 Maintenance Mapping

Table 13.2: Tool and Maintenance Mapping

Maintenance Type	**Tool**	**Example Usage**
Corrective	Loguru	Find application errors and events
Corrective	PySnooper	Trace incorrect calculations
Corrective	VizTracer	Inspect unexpected execution paths
Preventive	PySnooper	Understand code before refactoring
Preventive	VizTracer	Analyze execution behavior
Preventive	SnakeViz	Identify possible performance bottlenecks
Perfective	SnakeViz	Analyze newly added functionality
Adaptive	Loguru	Monitor new web functionality

The repository similarly maps Loguru, PySnooper, VizTracer, and SnakeViz to different maintenance activities. :contentReference[oaicite:14]index=14

14. Transformation Tool Simulation

14.1 Simulation in the Math Problem Solver

The tools were not considered only as theoretical concepts. They were applied to the Math Problem Solver project through dedicated analysis scripts.

The simulation can be represented as:

```
                    Math Problem Solver
                            |
          +-----------------+-----------------+
          |                 |                 |
       Loguru           PySnooper         VizTracer
          |                 |                 |
   Application         Execution          Execution
     Logging            Tracing          Visualization
                            |
                         SnakeViz
                            |
                    Performance Profiling
```

14.2 Loguru Simulation

The Loguru demonstration executes mathematical operations and records application events.

Example:

```
python analysis\logging_demo.py
```

Expected type of output:

```
[LOG] Addition operation performed
Addition result: 7.0
```

14.3 PySnooper Simulation

The PySnooper demonstration traces function execution.

```
python analysis\pysnooper_demo.py
```

Expected information includes:

```
Starting var: a = 4
Starting var: b = 3
New var: result = 7
Return value: 7
```

14.4 VizTracer Simulation

The VizTracer demonstration records execution information:

```
viztracer analysis\viztrace_demo.py
```

The generated visualization allows inspection of:

- Function execution order.
- Function duration.
- Execution timeline.
- Repeated calls.

14.5 SnakeViz Simulation

Profiling information is generated using:

```
python -m cProfile -o profile.prof analysis\profile_demo.py
```

The result is visualized using:

```
snakeviz profile.prof
```

The visualization provides function call counts, execution time, cumulative time, and potential performance bottlenecks.

14.6 Complete Simulation Sequence

The complete sequence from the project root is:

```
python main.py

python -m unittest discover -s tests -p "test_*.py"

python analysis\logging_demo.py

python analysis\pysnooper_demo.py

viztracer analysis\viztrace_demo.py

python -m cProfile -o profile.prof analysis\profile_demo.py

snakeviz profile.prof
```

The current repository documents this complete analysis command sequence. :contentReference[oaicite:15]index=15

15. Alternatives to the Selected Tools

15.1 Alternatives to Loguru

Possible alternatives include:

- Python built-in `logging`
- Structlog
- Logbook

The built-in Python `logging` module is a strong alternative because it is part of the Python standard library and does not require an additional external package.

15.2 Alternatives to PySnooper

Possible alternatives include:

- `pdb`
- VS Code Debugger
- Python `breakpoint()`

The VS Code Debugger is especially useful for interactive debugging because it supports breakpoints, variable inspection, call-stack inspection, and step-by-step execution.

15.3 Alternatives to VizTracer

Possible alternatives include:

- Pyinstrument
- cProfile
- Yappi

These tools can provide different forms of execution and performance analysis.

15.4 Alternatives to SnakeViz

Possible alternatives include:

- Pyinstrument
- Py-Spy
- Tuna

These tools provide alternative approaches to Python profiling and performance visualization.

The alternatives listed above are consistent with the alternatives documented in the project's current README. :contentReference[oaicite:16]index=16

16. Preferred Tools for Future Projects

16.1 Preferred Logging Tool: Loguru

For a small-to-medium Python application, I would prefer Loguru for application logging because of its simple syntax and readable output.

For larger systems with strict infrastructure requirements, Python's built-in logging framework may be preferable because of its standard library integration and extensive configuration options.

For the Math Problem Solver project, Loguru is convenient and easy to use.

16.2 Preferred Debugging Tool: VS Code Debugger

For interactive debugging, I would prefer the VS Code Debugger.

Important features include:

- Breakpoints.
- Variable inspection.
- Step-by-step execution.
- Call-stack inspection.
- Interactive debugging.

PySnooper is still valuable when a lightweight textual execution trace is preferred.

16.3 Preferred Runtime Analysis Tool: VizTracer

VizTracer is useful when a developer wants a visual representation of program execution.

It is particularly useful for understanding:

- Function relationships.
- Execution order.

- Runtime duration.
- Repeated operations.

16.4 Preferred Profiling Tool: SnakeViz

For projects using Python's `cProfile`, SnakeViz is a convenient tool for visualizing the resulting profiling data.

It makes profiling information easier to interpret than raw textual statistics.

16.5 Overall Preference

For a future Python software project, my preferred combination would be:

Table 16.1: Preferred Toolset

Purpose	**Preferred Tool**
Application Logging	Loguru
Interactive Debugging	VS Code Debugger
Runtime Execution Analysis	VizTracer
Profiling Visualization	SnakeViz
Automated Testing	unittest / pytest

The repository also identifies Loguru, VS Code Debugger, VizTracer, and SnakeViz as preferred tools for different future-project requirements. :contentReference[oaicite:17]index=17

17. Testing and Verification

17.1 Testing Framework

The project uses Python's `unittest` framework for automated testing.

The test suite covers the maintained mathematical functionality and integration behavior.

17.2 Test Command

The complete test suite is executed using:

```
python -m unittest discover -s tests -p "test_*.py"
```

17.3 Final Test Result

The final result was:

```
.................
----------------------------------------------------------------------

Ran 16 tests in 0.002s

OK
```

Therefore:

16 tests passed successfully

The repository records the final result as 16 successful automated tests. :contentReference[oaicite:18]index=18

17.4 Importance of Regression Testing

Regression testing is important after software maintenance because a change in one part of the application may unintentionally affect another part.

For example:

- Corrective maintenance required testing division behavior.
- Adaptive maintenance required ensuring that calculator functionality remained reusable.
- Preventive maintenance required checking validation and existing functionality.
- Perfective maintenance required testing newly added operations.

The successful final test suite provides evidence that the tested functionality remained operational after the maintenance changes.

18. Running the Project

18.1 Clone the Repository

The project repository is hosted on GitHub.

```
git clone https://github.com/Yasirraiyan/
210042152-SWE-4802-Software-Maintainance-Lab-Project.git
```

Then:

```
cd 210042152-SWE-4802-Software-Maintainance-Lab-Project
```

18.2 Create Virtual Environment

On Windows PowerShell:

```
python -m venv venv
```

Activate the virtual environment:

```
.\venv\Scripts\Activate.ps1
```

After successful activation, the terminal displays:

```
(venv)
```

18.3 Install Dependencies

The repository uses a `Requirements.txt` file.

From the project root:

```
python -m pip install -r Requirements.txt
```

18.4 Run Main Application

The main command-line application can be executed using:

```
python main.py
```

A typical interaction is:

```
===== Math Problem Solver =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Square Root
6. Cube Root
7. Power

Enter your choice: 1
Enter first number: 4
Enter second number: 3

Addition result: 7.0
```

The current repository documents the main application command and example output. :contentReference[oaicite:19]index=19

18.5 Run Tests

Use:

```
python -m unittest discover -s tests -p "test_*.py"
```

Expected final result:

```
................
----------------------------------------------------------------------

Ran 16 tests in 0.002s

OK
```

19. Importance of Analysis Tools in Maintenance

Software maintenance becomes easier when developers understand how an existing system behaves.

The selected tools provide different forms of evidence.

19.1 Loguru

Loguru provides evidence about application events.

$$\text{Loguru} \Rightarrow \text{What happened?}$$

19.2 PySnooper

PySnooper provides evidence about detailed function execution.

$$\text{PySnooper} \Rightarrow \text{How did the code execute?}$$

19.3 VizTracer

VizTracer provides evidence about execution sequence and timing.

$$\text{VizTracer} \Rightarrow \text{When and in what order did execution occur?}$$

19.4 SnakeViz

SnakeViz provides evidence about profiling and execution cost.

$$\text{SnakeViz} \Rightarrow \text{Where is execution time being spent?}$$

19.5 Maintenance Benefits

Together, the tools help developers:

- Debug existing defects.
- Understand unfamiliar code.
- Monitor application behavior.
- Trace execution paths.
- Find potential performance bottlenecks.
- Support refactoring.
- Improve maintainability.
- Validate changes.
- Support future software evolution.

20. Overall Project Status

The project evolved from a basic command-line mathematical application into a more complete and maintainable software system.

Table 20.1: Final Project Evolution

Version	**Major Change**
V1	Initial baseline mathematical application
V1.1	Corrective maintenance – division-by-zero handling
V1.2	Adaptive maintenance – Flask web interface
V1.3	Preventive maintenance – validation and maintainability improvements
V2.0	Perfective maintenance – additional mathematical features

20.1 Final Features

The final project provides:

- Addition.
- Subtraction.
- Multiplication.
- Division.
- Power.
- Square root.
- Cube root.

- Factorial.
- Percentage.
- Input validation.
- Error handling.
- Logging.
- Automated testing.
- Web adaptation.
- Execution analysis.
- Performance profiling.

The current repository identifies these features and analysis capabilities as part of the final project. :contentReference[oaicite:20]index=20

21. Discussion

The Math Problem Solver provides a practical example of how a relatively small software system can undergo multiple maintenance activities.

Corrective maintenance addressed an existing defect. Adaptive maintenance changed the environment in which the application could be used. Preventive maintenance improved internal software quality, and perfective maintenance added new user-oriented functionality.

The transformation and analysis tools complement these maintenance activities.

Loguru is useful for monitoring application events. PySnooper is useful for understanding detailed execution inside functions. VizTracer provides a visual representation of runtime behavior. SnakeViz provides an intuitive view of profiling information.

Using several tools is useful because each tool provides a different perspective.

For example:

$$\text{Loguru} + \text{PySnooper} + \text{VizTracer} + \text{SnakeViz}$$

provides a combination of:

- Application-level information.
- Function-level information.
- Runtime execution information.
- Performance information.

This combination supports software developers throughout the maintenance lifecycle.

22. Conclusion

This report presented the maintenance and analysis of the Math Problem Solver project.

The project demonstrated four major software maintenance categories.

First, corrective maintenance was used to fix the division-by-zero defect. Second, adaptive maintenance introduced a Flask-based web interface so that the application could operate in a web environment. Third, preventive maintenance improved validation, type information, error handling, code organization, and maintainability. Finally, perfective maintenance expanded the functionality of the application with additional mathematical operations.

The report also demonstrated four transformation and software analysis tools: Loguru, PySnooper, VizTracer, and SnakeViz.

Loguru provides application logging and helps identify what happened during execution. PySnooper provides detailed function-level tracing and helps explain how execution occurred. VizTracer provides a runtime execution timeline and helps developers understand program behavior. SnakeViz visualizes profiling information and helps identify possible performance bottlenecks.

The tools were applied to the Math Problem Solver rather than being considered only theoretically. Dedicated analysis scripts were used for logging, tracing, execution visualization, and profiling.

Finally, the project was verified through automated testing. The final test suite successfully passed 16 tests.

Therefore, the project successfully demonstrates how software can be maintained, analyzed, debugged, tested, improved, and extended throughout its lifecycle.

Bibliography

[1] Yasir Raiyan, *210042152-SWE-4802-Software-Maintainance-Lab-Project*, GitHub Repository, 2026.

https://github.com/Yasirraiyan/210042152-SWE-4802-Software-Maintainance-Lab-Projec

[2] Loguru Documentation, *Python Logging Made (Stupidly) Simple*.

https://loguru.readthedocs.io/

[3] PySnooper, *Python Debugging and Execution Tracing Tool*.

https://github.com/cool-RR/PySnooper

[4] VizTracer Documentation, *A Debugging and Profiling Tool for Python*.

https://viztracer.readthedocs.io/

[5] SnakeViz Documentation, *Web-based Viewer for Python Profiler Output*.

https://jiffyclub.github.io/snakeviz/

[6] Python Documentation, *The Python Standard Library*.

https://docs.python.org/3/library/

A. Complete Command Reference

A.1 Environment Setup

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install -r Requirements.txt
```

A.2 Run Application

```
python main.py
```

A.3 Run Tests

```
python -m unittest discover -s tests -p "test_*.py"
```

A.4 Run Loguru

```
python analysis\logging_demo.py
```

A.5 Run PySnooper

```
python analysis\pysnooper_demo.py
```

A.6 Run VizTracer

```
pip install viztracer
viztracer analysis\viztrace_demo.py
```

A.7 Run SnakeViz

```
pip install snakeviz
python -m cProfile -o profile.prof analysis\profile_demo.py
snakeviz profile.prof
```

A.8 Final Test Evidence

```
.................
----------------------------------------------------------------------


Ran 16 tests in 0.002s

OK
```